arXiv:2111.03794v2 [cs.LG] 14 Nov 2022

# Physics-Informed Neural Operator for Learning Partial Differential Equations

Zongyi Li*, Hongkai Zheng*, Nikola Kovachki, David Jin, Haoxuan Chen,
Burigede Liu, Kamyar Azizzadenesheli, Anima Anandkumar

November 15, 2022

**Abstract**

In this paper, we propose physics-informed neural operators (PINO) that uses available data and/or physics constraints to learn the solution operator of a family of parametric Partial Differential Equation (PDE). This hybrid approach allows PINO to overcome the limitations of purely data-driven and physics-based methods. For instance, data-driven methods fail to learn when data is of limited quantity and/or quality, and physics-based approaches fail to optimize on challenging PDE constraints. By combining both data and PDE constraints, PINO overcomes all these challenges. Additionally, a unique property that PINO enjoys over other hybrid learning methods is its ability to incorporate data and PDE constraints at different resolutions. This allows us to combine coarse-resolution data, which is inexpensive to obtain from numerical solvers, with higher resolution PDE constraints, and the resulting PINO has no degradation in accuracy even on high-resolution test instances. This discretization-invariance property in PINO is due to neural-operator framework which learns mappings between function spaces and allows evaluation at different resolutions without the need for re-training. Moreover, PINO succeeds in the purely physics setting, where no data is available, while other approaches such as the Physics-Informed Neural Network (PINN) fail due to optimization challenges, e.g. in multi-scale dynamic systems such as Kolmogorov flows. This is because PINO learns the solution operator by optimizing PDE constraints on multiple instances while PINN optimizes PDE constraints of a single PDE instance. Further, in PINO, we incorporate the Fourier neural operator (FNO) architecture which achieves orders-of-magnitude speedup over numerical solvers and also allows us to compute explicit gradients on function spaces efficiently.

## 1 Introduction

Machine learning methods have recently shown promise in solving partial differential equations (PDEs) [20, 23, 29, 3]. A recent breakthrough is the paradigm of operator learning for solving PDEs. Unlike standard neural networks that learn on inputs and outputs of a fixed dimension, neural operators learn operators, which are mappings between function spaces [20, 23, 24]. Neural operators, by design, are discretization-invariant, meaning at inference time, they can be evaluated at any data discretization or resolution without the need for retraining.

Neural operators are well suited for solving PDEs since they can learn the solution operator of a given family of parametric PDEs. Note that the solution operator is the mapping from the input function (initial and boundary conditions), to the output solution function. Previous works show that neural operators are able to capture complex multi-scale dynamic processes and are significantly faster than numerical solvers [20, 23, 27, 45, 34, 22, 44].

However, previous work assumes the availability of data to train neural operators, which can come either from existing numerical solvers or from direct observations of the physical phenomena. In many scenarios, such data can be expensive to generate. The training dataset is unavailable or available only as low resolution observations [13]. This limits the ability of neural operators to learn high-fidelity models. Moreover, generalization of the learned neural operators to unseen scenarios and conditions that are different from training data is challenging.

An alternative to data-driven approaches for solving PDEs is physics-based and it requires no training data. A popular framework known as Physics-Informed Neural Network (PINN) [35] uses optimization to find the solution function of a given PDE instance. PINN uses a neural network as the ansatz of the solution function and optimizes a loss function to minimize violation of the given equation by taking advantage of auto-differentiation to compute the exact, mesh-free derivatives. PINN overcomes the need to choose a discretization grid that numerical solvers require, e.g. finite difference methods (FDM) and finite element methods (FEM). It has shown promise in solving PDEs in many

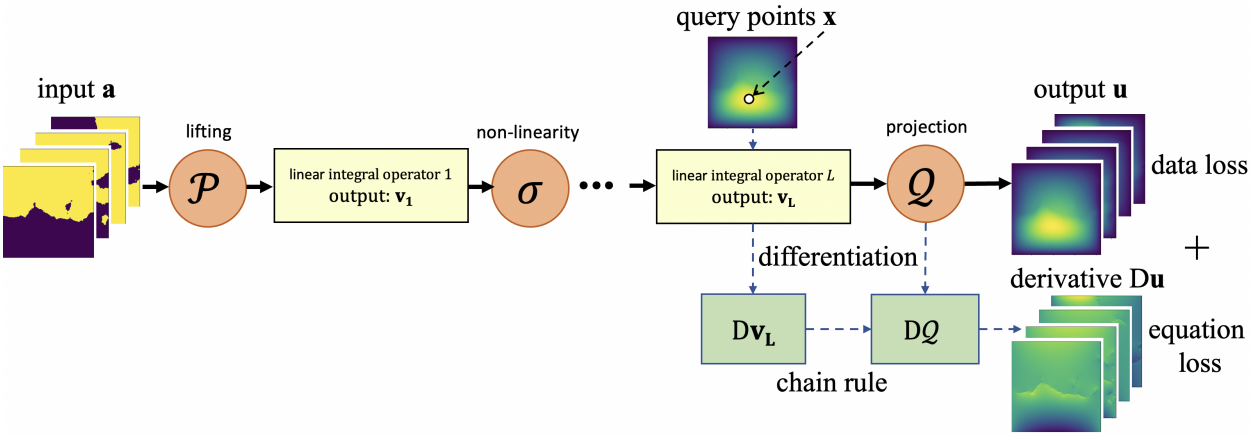



Figure 1: PINO trains neural operator with both the data loss and equation loss. The figure shows the neural operator architecture with the lifting point-wise operator that receives input function $a$ and outputs function $v_0$ with a larger co-dimension. This operation is followed by $L$ blocks that compute linear integral operators followed by non-linearity, and the last layer of which outputs the function $v_L$. The pointwise projection operator projects $v_L$ to output function $u$. Both $v_L$ and $u$ are functions and all their derivatives ($\mathrm{D}v_\mathrm{L}$, $\mathrm{D}u$) can be computed in their exact forms at any query points $x$.

systems include inverse and higher dimensional problems. [31, 10, 12, 17]. Recently, researchers have developed many variations of PINN with promising results on inverse problems and partially observed tasks [28, 47, 38].

However, PINN fails in many multi-scale dynamic PDE systems [42, 7, 36] due to two main reasons, viz., (1) the challenging optimization landscape of the PDE constraints [40] and its sensitivity to hyper-parameter selection [39], and (2) the difficulty to propagate information from the initial or boundary conditions to unseen parts of the interior or to future times [6]. Moreover, PINN only learns the solution function of a single PDE instance, and cannot generalize to other instances without re-optimization. Concurrent work on physics-informed DeepONet that imposes PDE losses on operator learning [41] overcomes this limitation and can learn across multiple instances. While the PDE loss is computed at any query points, the input sensors limits to a fixed grid or basis, and therefore it is not discretization invariant [20], and its architecture comes with limitations of linear approximation [21]. We show none of the mentioned limitations are present in the current paper.

**Our contributions.** To overcome the above shortcomings of purely physics-informed optimization or data-driven learning, we propose the physics-informed neural operator (PINO). It utilizes both the data and equation constraints (whichever are available) for operator learning. To further improve the accuracy of PINO, we fine-tune the learned operator on a given test instance. This allows us to overcome the limitation of the learned operators to have an arbitrarily low error on a given test instance. A schematic of PINO is shown in Figure 1, where the neural operator architecture consists of linear integral operators followed by non-linearity. This composition with non-linearity gives neural operators the expressivity to capture any non-linear continuous operators. The derivatives needed for the equation loss in PINO are computed explicitly through the operator layers in function spaces.

A unique feature that PINO enjoys over other hybrid learning methods [47, 46, 14] is its **resolution-invariant** property to incorporate data and PDE constraints at different resolutions. This allows us to combine coarse-resolution training data with higher resolution PDE constraints. We show that the PINO model learned from such multi-resolution hybrid loss functions has almost no degradation in accuracy even on high-resolution test instances, when only low-resolution training data is available. Additionally, in many cases, we are able to learn the solution operator with PINO even when no training data is available while other physics-based approaches like PINN and PI-DeepONet fail to converge.

The equation constraints in PINO vastly improve **generalization** and physical validity in operator learning compared to purely data-driven methods. PINO requires fewer to no training data and generalizes better compared to the data-driven FNO [25], especially on high-resolution test instances. On average, the relative error is 7% lower on the transient and Kolmogorov flows, while matching the speedup of data trained FNO architecture (400x) compared to

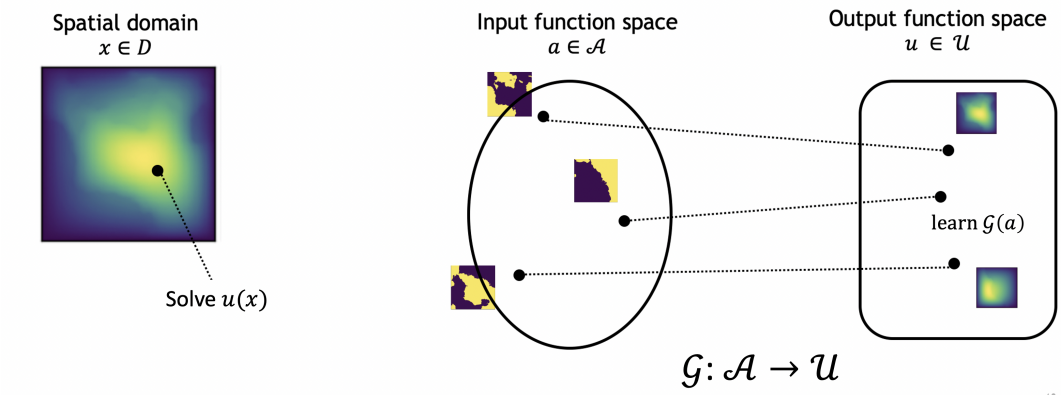



Figure 2: solve for one specific instance verse learn the entire solution operator
Left: numerical solvers and PINNs focus on solving one specific instance. Right: neural operators learn the solution operator for a family of equations.

the GPU-based pseudo-spectral solver [11]. Further, the PINO model on the Navier Stokes equation can be easily transferred to different Reynolds numbers ranging from 100 to 500 using instance-wise fine-tuning.

The learned operator ansatz helps PINO overcomes the **optimization** challenge of the physics-informed learning. We efficiently compute the explicit gradients on function space through Fourier space computations. In contrast, previous auto-differentiation methods has to compute the derivatives at sampling locations. Even on a single instance, PINO has 20x smaller error and 25x speedup on the chaotic Kolmogorov flow, demonstrating a better optimization landscape of the neural operators over standard neural networks.

We also use PINO for **inverse problems** through two approaches: (1) learning the forward solution operator and differentiating through it to get the inverse solution, or (2) learning the inverse solution operator directly. Imposing the PDE loss guarantees the inverse solution is physically invalid in both approaches. We find that of these two approaches, the latter is more accurate for recovering the coefficient function in the Darcy flow and it is 3000x faster than the conventional solvers using accelerated MCMC [5].

## 2 Preliminaries and problem settings

In this section, we first define the stationary and dynamic PDE systems that we consider. We give an overview of the physics-informed setting and operator-learning setting. In the end, we define the Fourier neural operator as a specific model for operator learning.

### 2.1 Problem settings

We consider two natural class of PDEs. In the first, we consider the stationary system

$$\begin{aligned} \mathcal{P}(u, a) &= 0, && \text{in } D \subset \mathbb{R}^d \\ u &= g, && \text{in } \partial D \end{aligned} \tag{1}$$

where $D$ is a bounded domain, $a \in \mathcal{A} \subseteq \mathcal{V}$ is a PDE coefficient/parameter, $u \in \mathcal{U}$ is the unknown, and $\mathcal{P} : \mathcal{U} \times \mathcal{A} \to \mathcal{F}$ is a possibly non-linear partial differential operator with $(\mathcal{U}, \mathcal{V}, \mathcal{F})$ a triplet of Banach spaces. Usually the function $g$ is a fixed boundary condition (potentially can be entered as a parameter). This formulation gives rise to the solution operator $\mathcal{G}^\dagger : \mathcal{A} \to \mathcal{U}$ defined by $a \mapsto u$. A prototypical example is the second-order elliptic equation $\mathcal{P}(u, a) = -\nabla \cdot (a \nabla u) + f$.

In the second setting, we consider the dynamical system

$$\begin{aligned} \frac{du}{dt} &= \mathcal{R}(u), && \text{in } D \times (0, \infty) \\ u &= g, && \text{in } \partial D \times (0, \infty) \\ u &= a && \text{in } \bar{D} \times \{0\} \end{aligned} \tag{2}$$

where $a = u(0) \in \mathcal{A} \subseteq \mathcal{V}$ is the initial condition, $u(t) \in \mathcal{U}$ for $t > 0$ is the unknown, and $\mathcal{R}$ is a possibly non-linear partial differential operator with $\mathcal{U}$, and $\mathcal{V}$ Banach spaces. As before, we take $g$ to be a known boundary condition. We assume that $u$ exists and is bounded for all time and for every $u_0 \in \mathcal{U}$. This formulation gives rise to the solution operator $\mathcal{G}^\dagger : \mathcal{A} \to C\big((0,T];\mathcal{U}\big)$ defined by $a \mapsto u$. Prototypical examples include the Burgers' equation and the Navier-Stokes equation.

## 2.2 Solving equation using the physics-informed neural networks

Given an instance $a$ and a solution operator $\mathcal{G}^\dagger$ defined by equations (1) or (2) , we denote by $u^\dagger = \mathcal{G}^\dagger(a)$ the unique ground truth. The equation solving task is to approximate $u^\dagger$. This setting consists of the ML-enhanced conventional solvers such as learned finite element, finite difference, and multigrid solvers [19, 33, 9], as well as purely neural network-based solvers such as the Physics-Informed Neural Networks (PINNs), Deep Galerkin Method, and Deep Ritz Method [35, 37, 43]. Especially, these PINN-type methods use a neural network $u_\theta$ with parameters $\theta$ as the the ansatz to approximate the solution function $u^\dagger$ . The parameters $\theta$ are found by minimizing the physics-informed loss with exact derivatives computed using automatic-differentiation (autograd). In the stationary case, the physics-informed loss is defined by minimizing the l.h.s. of equation (1) in the squared norm of $\mathcal{F}$. A typical choice is $\mathcal{F} = L^2(D)$, giving the loss function

$$\begin{aligned}\mathcal{L}_{\text{pde}}(a, u_\theta) &= \Big\|\mathcal{P}(a, u_\theta)\Big\|^2_{L^2(D)} + \alpha\Big\|u_\theta|_{\partial D} - g\Big\|^2_{L^2(\partial D)} \\ &= \int_D |\mathcal{P}(u_\theta(x), a(x))|^2 \mathrm{d}x + \alpha \int_{\partial D} |u_\theta(x) - g(x)|^2 \mathrm{d}x\end{aligned} \tag{3}$$

In the case of a dynamical system, it minimizes the residual of equation (2) in some natural norm up to a fixed final time $T > 0$. A typical choice is the $L^2\big((0,T]; L^2(D)\big)$ norm, yielding

$$\begin{aligned}\mathcal{L}_{\text{pde}}(a, u_\theta) &= \Big\|\frac{du_\theta}{dt} - \mathcal{R}(u_\theta)\Big\|^2_{L^2(T;D)} + \alpha\Big\|u_\theta|_{\partial D} - g\Big\|^2_{L^2(T;\partial D)} + \beta\Big\|u_\theta|_{t=0} - a\Big\|^2_{L^2(D)} \\ &= \int_0^T \int_D |\frac{du_\theta}{dt}(t,x) - \mathcal{R}(u_\theta)(t,x)|^2 \mathrm{d}x\mathrm{d}t \\ &\quad + \alpha \int_0^T \int_{\partial D} |u_\theta(t,x) - g(t,x)|^2 \mathrm{d}x\mathrm{d}t \\ &\quad + \beta \int_D |u_\theta(0,x) - a(x)|^2 \mathrm{d}x\end{aligned} \tag{4}$$

The PDE loss consists of the physics loss in the interior and the data loss on the boundary and initial conditions, with hyper-parameters $\alpha, \beta > 0$. It can be generalized to variational form as in [43].

**Challenges of PINN.** PINNs take advantage of the universal approximability of neural networks, but, in return, suffer from the low-frequency induced bias. Empirically, PINNs often fail to solve challenging PDEs when the solution exhibits high-frequency or multi-scale structure [40, 42, 7, 36]. Further, as an iterative solver, PINNs have difficulty propagating information from the initial condition or boundary condition to unseen parts of the interior or to future times [6]. For example, in challenging problems such as turbulence, PINNs are only able to solve the PDE on a relatively small domain [16], or otherwise, require extra observational data which is not always available in practice [36, 4]. In this work, we propose to overcome the challenges posed by the optimization by integrating operator learning with PINNs.

## 2.3 Learning the solution operator via neural operator

An alternative setting is to learn the solution operator $\mathcal{G}$. Given a PDE as defined in (1) or (2) and the corresponding solution operator $\mathcal{G}^\dagger$, one can use a neural operator $\mathcal{G}_\theta$ with parameters $\theta$ as a surrogate model to approximate $\mathcal{G}^\dagger$. Usually we assume a dataset $\{a_j, u_j\}_{j=1}^N$ is available, where $\mathcal{G}^\dagger(a_j) = u_j$ and $a_j \sim \mu$ are i.i.d. samples from some distribution $\mu$ supported on $\mathcal{A}$. In this case, one can optimize the solution operator by minimizing the empirical data

loss on a given data pair

$$\mathcal{L}_{\text{data}}(u, \mathcal{G}_\theta(a)) = \|u - \mathcal{G}_\theta(a)\|^2_{\mathcal{U}} = \int_D |u(x) - \mathcal{G}_\theta(a)(x)|^2 \mathrm{d}x \tag{5}$$

where we assume the setting of (1) for simplicity of the exposition. The operator data loss is defined as the average error across all possible inputs

$$\mathcal{J}_{\text{data}}(\mathcal{G}_\theta) = \|\mathcal{G}^\dagger - \mathcal{G}_\theta\|^2_{L^2_\mu(\mathcal{A};\mathcal{U})} = \mathbb{E}_{a\sim\mu}[\mathcal{L}_{\text{data}}(a, \theta)] \approx \frac{1}{N}\sum_{j=1}^{N} \int_D |u_j(x) - \mathcal{G}_\theta(a_j)(x)|^2 \mathrm{d}x. \tag{6}$$

Similarly, one can define the operator PDE loss as

$$\mathcal{J}_{\text{pde}}(\mathcal{G}_\theta) = \mathbb{E}_{a\sim\mu}[\mathcal{L}_{\text{pde}}(a, \mathcal{G}_\theta(a))]. \tag{7}$$

In general, it is non-trivial to compute the derivatives $d\mathcal{G}_\theta(a)/dx$ and $d\mathcal{G}_\theta(a)/dt$ for model $\mathcal{G}_\theta$. In the following section, we will discuss how to compute these derivatives for Fourier neural operator.

## 2.4 Neural operators

In this work, we will focus on the neural operator model designed for the operator learning problem [25]. The neural operator is formulated as a generalization of standard deep neural networks to operator setting. Neural operator composes linear integral operator $\mathcal{K}$ with pointwise non-linear activation function $\sigma$ to approximate highly non-linear operators.

**Definition 1 (Neural operator $\mathcal{G}_\theta$)** *Define the neural operator*

$$\mathcal{G}_\theta := \mathcal{Q} \circ (\mathcal{W}_L + \mathcal{K}_L) \circ \cdots \circ \sigma(\mathcal{W}_1 + \mathcal{K}_1) \circ \mathcal{P} \tag{8}$$

*where $\mathcal{P}$ and $\mathcal{Q}$ are pointwise operators, parameterized with neural networks $P : \mathbb{R}^{d_a} \to \mathbb{R}^{d_1}$ and $Q : \mathbb{R}^{d_L} \to \mathbb{R}^{d_u}$, where $d_a$ is the co-dimension of an input function $a \in A$ and $d_u$ is the co-dimension of the output function $u$. $\mathcal{P}$ operator lifts the lower dimension function into higher dimensional space and $\mathcal{Q}$ operator projects the higher dimension function back to the lower dimensional space. The model stacks $L$ layers of $\sigma(\mathcal{W}_l + \mathcal{K}_l)$ where $\mathcal{W}$ are pointwise linear operators parameterized as matrices $W_l \in \mathbb{R}^{d_{l+1}\times d_l}$ , $\mathcal{K}_l : \{D \to \mathbb{R}^{d_l}\} \to \{D \to \mathbb{R}^{d_{l+1}}\}$ are integral kernel operators, and $\sigma$ are fixed activation functions. The parameters $\theta$ consists of all the parameters in $\mathcal{P}, \mathcal{Q}, \mathcal{W}_l, \mathcal{K}_l$.*

**Definition 2 (Kernel Integral Operators)** *We define the kernel integral operator $\mathcal{K}$ used in* (8)*. Let $\kappa^{(l)} \in C(D \times D; \mathbb{R}^{d_{l+1}\times d_l})$ and let $\nu$ be a Borel measure on $D$. Then we define $\mathcal{K}$ by*

$$(\mathcal{K}v_l)(x) = \int_D \kappa^{(l)}(x, y)v_l(y)\, d\nu(y) \qquad \forall x \in D. \tag{9}$$

The kernel integral operator can be discretized and implemented with graph neural networks as in graph neural operators **(author?)** [25].

$$(\mathcal{K}v_l)(x) = \sum_{B(x)} \kappa^{(l)}(x, y)v_l(y) \qquad \forall x \in D. \tag{10}$$

where $B(x)$ is a ball of center at $x$. As a generalization, the kernel function can also take the form of $(\mathcal{K}v_l)(x) = \sum_{B(x)} \kappa^{(l)}(x, y, v_l(y))$.

Recently, **(author?)** [23] proposes the Fourier neural operator (FNO) that restricts the integral operator $\mathcal{K}$ to convolution. In this case, it can apply the Fast Fourier Transform (FFT) to efficiently compute $\mathcal{K}$. This leads to a fast architecture that obtains state-of-the-art results for PDE problems.

**Definition 3 (Fourier convolution operator)** *One specific form of the kernel integral operators is the Fourier convolution operator*

$$\big(\mathcal{K}v_l\big)(x) = \mathcal{F}^{-1}\Big(R \cdot (\mathcal{F}v_l)\Big)(x) \qquad \forall x \in D \tag{11}$$

*where $\mathcal{F}, \mathcal{F}^{-1}$ are the Fast Fourier transform and its inverse; $R$ is part of the parameter $\theta$ to be learn.*

One can build a neural operator with mixed kernel integral layers and Fourier convolution layers. If the input and output query points are sampled from non-uniform mesh, we can use the graph kernel operator as the first and last layer for continuous evaluation, while using the Fourier layers in the middle for efficient computation, similar to [22].

**Challenges of operator learning.** Operator learning is similar to supervised learning in computer vision and language where data play a very important role. One needs to assume the training points and testing points follow the same problem setting and the same distribution. Especially, the previous FNO model trained on one coefficient (e.g. Reynolds number) or one geometry cannot be easily generalized to another. Moreover, for more challenging PDEs where the solver is very slow or the solver is even not existent, it is hard to gather a representative dataset. On the other hand, since prior training methods for FNO do not use any knowledge of the equation, the trained models cannot get arbitrarily close to the ground truth by using the higher resolution as in conventional solvers, leaving a gap of generalization error. These challenges limit the applications of the prior works beyond accelerating the solver and modeling real-world experiments. In the following section, we will introduce the PINO framework to overcome these problems by using the equation constraints.

# 3 Physics-informed neural operator (PINO)

We propose the PINO framework that uses one neural operator model $\mathcal{G}_\theta$ for solving both operator learning problems and equation solving problems. It consists of two phases

- **operator learning**: learn a neural operator $\mathcal{G}_\theta$ to approximate the target solution operator $\mathcal{G}^\dagger$ using either/both the data loss $\mathcal{J}_{data}$ or/and the PDE loss $\mathcal{J}_{pde}$.
- **instance-wise fine-tuning**: use $\mathcal{G}_\theta(a)$ as the ansatz to approximate $u^\dagger$ with the pde loss $\mathcal{L}_{pde}$ and/or an additional operator loss $\mathcal{L}_{op}$ obtained from the operator learning phase.

## 3.1 Physics-informed operator learning

For operator learning, we use the physics constraints $\mathcal{J}_{pde}$ and supervision from data to train the neural operator. Especially one can sample unlimited amount of virtual PDE instances by drawing additional initial conditions or coefficient conditions $a_j \sim \mu$ for training. In this sense, we have access to the unlimited dataset by sampling new input $a_j$ in each training iteration. This advantage of using PDE constraints removes the requirement on the dataset and tune the supervised problem semi-supervised.

While PINO can be trained with physics constraints $\mathcal{J}_{pde}$ only, the $\mathcal{J}_{data}$ can provide stronger supervision than physics constraints and thus make the optimization much easier. PINO leverages the supervision from any available data to combine with physics constraints for better optimization landscape and thus learning accurate neural operators. A special case is to train neural operator use low-resolution data instances with high-resolution pde constraint, which will be studied in section 4.

## 3.2 Instance-wise fine-tuning of trained operator ansatz

Given a learned operator $\mathcal{G}_\theta$, we use $\mathcal{G}_\theta(a)$ as the ansatz to solve for $u^\dagger$. The optimization procedure is similar to PINNs where it computes the PDE loss $\mathcal{L}_{pde}$ on $a$, except that we propose to use a neural operator instead of a neural network. Since the PDE loss is a soft constraint and challenging to optimize, we also add an optional operator loss $\mathcal{L}_{op}$ (anchor loss) to bound the further fine-tuned model from the learned operator model

$$\mathcal{L}_{op}\big(\mathcal{G}_{\theta_i}(a), \mathcal{G}_{\theta_0}(a)\big) \coloneqq \|\mathcal{G}_{\theta_i}(a) - \mathcal{G}_{\theta_0}(a)\|^2_{\mathcal{U}}$$

where $\mathcal{G}_{\theta_i}(a)$ is the model at $i^{th}$ training epoch. We update the operator $\mathcal{G}_\theta$ using the loss $\mathcal{L}_{pde} + \alpha\mathcal{L}_{op}$. It is possible to further apply optimization techniques to fine-tune the last fewer layers of the neural operator and progressive training that gradually increase the grid resolution and use finer resolution in test time.

**Optimization landscape.** Using the operator as the ansatz has two major advantages: (1) PINN does point-wise optimization, while PINO does optimization in the space of functions. In the linear integral operation $\mathcal{K}$, the operator parameterizes the solution function as a sum of the basis function. Optimization of the set of coefficients and basis is easier than just optimizing a single function as in PINNs. (2) we can learn these basis functions in the operator learning phase which makes the later instance-wise fine-tuning even easier. In PINO, we do not need to propagate the information from the initial condition and boundary condition to the interior. It just requires fine-tuning the solution function parameterized by the solution operator.

**Trade-off.** (1) complexity and accuracy: instance-wise fine-tuning is an option to spend more computation in exchange for better accuracy. The learned operator is extremely fast since it is performing inference on the neural operator. On the other hand, instance-wise fine-tuning can improve accuracy while incurring more computational cost. (2) resolution effects on optimization landscape and truncation error (i.e. the error of the numerical differentiation): using a higher resolution and finer grid will reduce the truncation error. However, it has a higher computational complexity and memory consumption. A higher resolution may also potentially make the optimization unstable. Using hard constraints such as the anchor loss $\mathcal{L}_{op}$ relieves such a problem.

### 3.3 Derivatives of neural operators

In order to use the equation loss $\mathcal{L}_{pde}$, one of the major technical challenge is to efficiently compute the derivatives $D(\mathcal{G}_\theta a) = d(\mathcal{G}_\theta a)/dx$ for neural operators. In this section, we discuss three efficient methods to compute the derivatives of neural operator $\mathcal{G}_\theta$ as defined in (8).

**Numerical differentiation.** A simple but efficient approach is to use conventional numerical derivatives such as finite difference and Fourier differentiation [47, 8]. These numerical differentiation methods are fast and memory-efficient: given a $n$-points grid, finite difference requires $O(n)$, and the Fourier method requires $O(n \log n)$. These methods are agnostic to the underlying more. It can be applied to the neural operator with Graph layer 2 or Fourier layer 3 or neural networks such as UNet.

However, the numerical differentiation methods face the same challenges as the corresponding numerical solvers: finite difference methods require a fine-resolution uniform grid; spectral methods require smoothness and uniform grids. Especially. These numerical errors on the derivatives will be amplified on the output solution.

**Pointwise differentiation with autograd.** Similar to PINN [35], the most general way to compute the exact derivatives is to use the auto-differentiation library of neural networks (autograd). To apply autograd, one needs to use a neural network to parameterize the solution function $u : x \mapsto u(x)$. However, it is not straightforward to write out the solution function in the neural operator which directly outputs the numerical solution $u = \mathcal{G}_\theta(a)$ on a grid, especially for FNO which uses FFT. To apply autograd, we design a query function $u$ that input $x$ and output $u(x)$. Recall $\mathcal{G}_\theta \coloneqq \mathcal{Q} \circ (\mathcal{W}_L + \mathcal{K}_L) \circ \cdots \circ \sigma(\mathcal{W}_1 + \mathcal{K}_1) \circ \mathcal{P}$ and $u = \mathcal{G}_\theta a = \mathcal{Q} v_L = \mathcal{Q}(\mathcal{W}_L + \mathcal{K}_L) v_{L-1} \ldots$. Since $\mathcal{Q}$ is pointwise,

$$u(x) = \mathcal{Q}(v_L)(x) = Q(v_L(x)) = Q\big((\mathcal{W}_L v_{L-1})(x) + \mathcal{K}_L v_{L-1}(x)\big) \tag{12}$$

For both the kernel operator and Fourier operator, we either remove the pointwise residual term of the last layer $(\mathcal{W}_L v_{L-1})(x)$ or define the query function as an interpolation function on $\mathcal{W}_L$.

For kernel integral operator 2, the kernel function can directly take the query points are input. So the query function

$$u(x) = Q\big(\sum_{B(x)} \kappa^{(l)}(x, y, v_{L-1}(y))\big)$$

where we omit the derivative of the support $B(x)$. We can apply auto-differentiation to compute the derivatives

$$u'(x) = Q'\big(v_L(x)\big) \cdot \sum_{B(x)} \kappa^{(l)\prime}(x, y, v_{L-1}(y)) \tag{13}$$

Similarly, for the Fourier convolution operator, we need to evaluate the Fourier convolution $\mathcal{K}_L v_{L-1}(x)$ on the query points $x$. It can be done by writing out the output function as Fourier series composing with $Q$ :

$$u(x) = \mathcal{Q} \circ \mathcal{F}^{-1}\Big(R \cdot (\mathcal{F} v_{L-1}))\Big)(x) = Q\Big(\frac{1}{k_{max}} \sum_{k=0}^{k_{max}} \big(R_k (\mathcal{F} v_{L-1})_k\big) \exp \frac{i2\pi k}{D}(x)\Big)$$

Where $\mathcal{F}$ is the discrete Fourier transform. The inverse discrete Fourier transform is the sum of $k_{max}$ Fourier series with the coefficients $\big(R_k(\mathcal{F} v_{L-1})_k\big)$ coming from the previous layer.

$$u'(x) = Q'\big(v_L(x)\big) \cdot \frac{1}{k_{max}} \sum_{k=0}^{k_{max}} \big(R_k (\mathcal{F} v_{L-1})_k\big) \exp' \frac{i2\pi k}{D}(x) \tag{14}$$

Notice $\exp' \frac{i2\pi k}{D}(x) = \frac{i2\pi k}{D} \exp \frac{i2\pi k}{D}(x)$, just as the numerical Fourier method. If the query points $x$ are a uniform grid, the derivative can be efficiently computed with the Fast Fourier transform.

The autograd method is general and exact, however, it is less efficient. Since the number of parameters $|\theta|$ is usually much greater than the grid size $n$, the numerical methods are indeed significantly faster. Empirically, the autograd method is usually slower and memory-consuming.

**Function-wise differentiation.** While it is expensive to apply the auto differentiation per query point, the derivative can be batched and computed in a function-wise manner. We develop an efficient and exact derivatives method based on the architecture of the neural operator that can compute the full gradient field. The idea is similar to the autograd, but we explicitly write out the derivatives on the Fourier space and apply the chain rule. Given the explicit form (14), $u'$ can be directly computed on the Fourier space.

$$u' = \mathcal{Q}'\big(v_L\big) \cdot \mathcal{F}^{-1}\Big(\frac{i2\pi}{D} K \cdot (\mathcal{F} v_L)\Big)\Big) \tag{15}$$

Therefore, to exactly compute the derivative of the Fourier neural operator, one just needs to run the numerical Fourier differentiation. Especially the derivative and be efficiently computed with the Fast Fourier transform when the query is uniform. Similarly, if the kernel function $\kappa^{(l)}$ in (13) has a structured form, we can also write out its the gradient field explicit.

Next, we show how to compute higher orders derivatives in their exact form, without evoking the autograd method. To this end, we can directly apply the chain rule for the higher-order derivatives without calling autograd. For example, the first order derivatives is $u' = (Q \circ v_L)' = \mathcal{Q}'(v_L) \cdot v_L'$ and the 2nd-order derivatives is $u'' = (\mathcal{Q} v_L)'' = v_L'^2 \cdot \mathcal{Q}''(v_L) + \mathcal{Q}'(v_L) \cdot v_L''$. Higher order derivatives can be similarly computed using chain rule. Furthermore, derivative based quantities on $v_L$, e.g., $v_L'$ can be computed in its exact form in the Fourier domain. Similarly, we can write out the higher-order derivatives of $Q$ using chain rule. Usually $Q$ is parameterized as a two layer neural networks $Q(x) = (A_2 \sigma(A_1 x + b_1) + b_2)$. So $Q'(x) = A_2 \sigma'(A_1 x + b_1) A_1$. In this manner, we have got the explicit formula of the derivatives for all neural operators.

**Fourier continuation.** The Fourier method has its best performance when applying on periodic problems. If the target function is non-periodic or non-smooth, the Fourier differentiation is not accurate. To deal with this issue, we apply the Fourier continuation method that embed the problem domain into a larger and periodic space. The extension can be simply done by padding zeros in the input. The loss is computed at the original space during training. The Fourier neural operator will automatically generate a smooth extension. The details are given in Appendix C.

## 3.4 Inverse problem

The physics-informed method can be use to solve the inverse problem, where given the output function $u$, the goal is to recover (a distribution of) the input function $a$. By imposing the constraint $\mathcal{P}(u, a) = 0$, we can restrict $a$ to a physically-valid manifold. We propose two formulations to do the optimization-based inverse problem with PINO: the forward operator model and the inverse operator model.

- **Forward model**: learn the forward operator $\mathcal{G}_\theta : a \mapsto u$ with data. Initialize $\hat{a}$ to approximate $a^\dagger$. Optimize $\hat{a}$ using

$$\mathcal{J}_{forward} \coloneqq \mathcal{L}_{pde}(\hat{a}, u^\dagger) + \mathcal{L}_{data}(\mathcal{G}_\theta(\hat{a})) + R(\hat{a}). \tag{16}$$

- **inverse model**: learn the inverse operator $\mathcal{F}_\theta : u \mapsto a$ with data. Use $\mathcal{F}_\theta(u^\dagger)$ to approximate $a^\dagger$. Optimize $\mathcal{F}_\theta$ using

$$\mathcal{J}_{backward} \coloneqq \mathcal{L}_{pde}(\mathcal{F}_\theta(u^\dagger), u^\dagger) + \mathcal{L}_{op}(\mathcal{F}_\theta(u^\dagger), \mathcal{F}_{\theta_0}(u^\dagger)) + R(\mathcal{F}_\theta(u^\dagger) \tag{17}$$

Where $\mathcal{L}_{pde}$ is the PDE loss; $\mathcal{L}_{op}$ is the operator loss from the learned operator; $R(a)$ is the regularization term. We use the PDE loss $\mathcal{L}_{pde}$ to deal with the small error in $\mathcal{G}_\theta$ and the ill-defining issue of $\mathcal{F}_\theta$. We provide a numerical study in section 4.3.

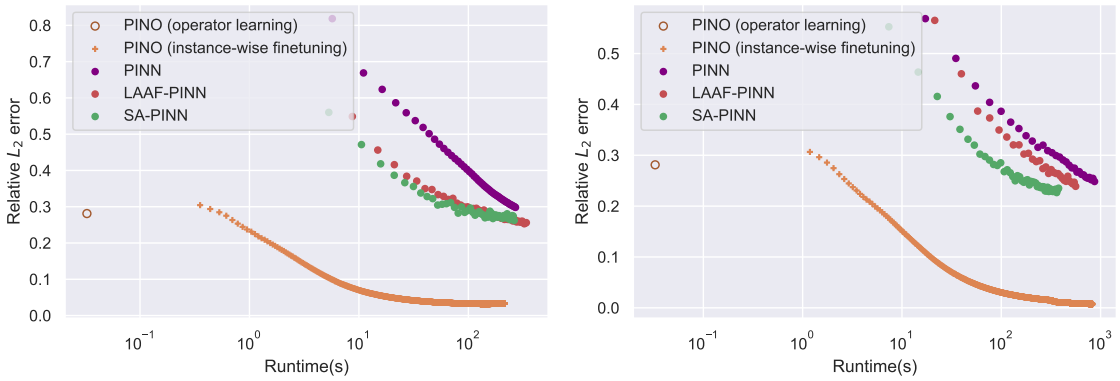



Figure 3: Plot of test relative $L_2$ error versus runtime step for the Kolmogorov flow with Re500, T=0.5s. Left: resolution $64 \times 64 \times 65$; right: resolution $128 \times 128 \times 129$. Averaged over 20 instances. LAAF-PINN: PINN with locally adaptive activation functions. SA-PINN: self-adaptive PINN.

# 4 Experiments

In this section, we conduct empirical experiments to examine the efficacy of the proposed PINO. We present the PDE settings, their domains, and function spaces. In 4.1, we show using PDE constraint in operator learning results in neural operators that (1) generalize to very high resolution unseen data. (2) achieve smaller generalization errors with fewer to no data. Then in 4.2, we investigate how PINO uses the operator ansatz to solve harder equations with improved speed and accuracy. We study three concrete cases of PDEs on Burgers' Equation, Darcy Equation, and Navier-Stokes equation. In 4.3 we study the inverse problems. The implementation details of PINO and baseline methods are listed in Appendix A.

**Burgers' Equation.** The 1-d Burgers' equation is a non-linear PDE with periodic boundary conditions where $u_0 \in L^2_{\text{per}}((0,1);\mathbb{R})$ is the initial condition and $\nu = 0.01$ is the viscosity coefficient. We aim to learn the operator mapping the initial condition to the solution, $\mathcal{G}^\dagger : u_0 \mapsto u|_{[0,1]}$.

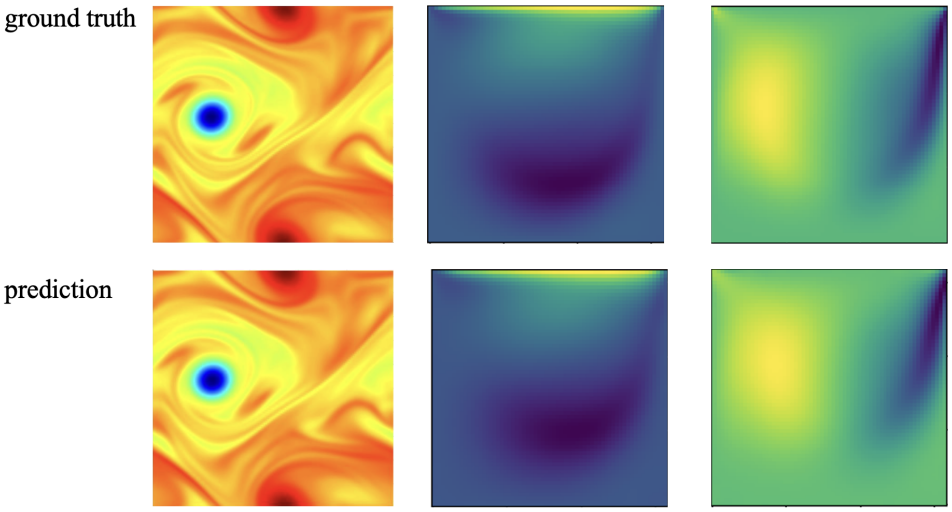
$$\begin{aligned} \partial_t u(x,t) + \partial_x (u^2(x,t)/2) &= \nu \partial_{xx} u(x,t), \qquad && x \in (0,1), t \in (0,1] \\ u(x,0) &= u_0(x), && x \in (0,1) \end{aligned} \tag{18}$$

**Darcy Flow.** The 2-d steady-state Darcy Flow equation on the unit box which is the second order linear elliptic PDE with a Dirichlet boundary where $a \in L^\infty((0,1)^2;\mathbb{R}_+)$ is a piecewise constant diffusion coefficient and $f = 1$ is a fixed forcing function. We are interested in learning the operator mapping the diffusion coefficient to the solution, $\mathcal{G}^\dagger : a \mapsto u$. Note that although the PDE is linear, the operator $\mathcal{G}^\dagger$ is not.

$$\begin{aligned} -\nabla \cdot (a(x) \nabla u(x)) &= f(x) \qquad && x \in (0,1)^2 \\ u(x) &= 0 && x \in \partial(0,1)^2 \end{aligned} \tag{19}$$

Since $a$ is in $L_{inf}$, we considered both the strong form $\mathcal{L}_{pde}(u) = \nabla \cdot (a \nabla u) - f$ and the weak form minimization loss $\mathcal{L}_{pde}(u) = -\frac{1}{2}(a \nabla u, \nabla u) - (u, f), u \in H_1$. Experiments show the strong form has a better performance.

**Navier-Stokes Equation.** We consider the 2-d Navier-Stokes equation for a viscous, incompressible fluid in vorticity form on the unit torus, where $u \in C([0,T]; H^r_{\text{per}}((0,l)^2;\mathbb{R}^2))$ for any $r > 0$ is the velocity field, $w = \nabla \times u$ is the vorticity, $w_0 \in L^2_{\text{per}}((0,l)^2;\mathbb{R})$ is the initial vorticity, $\nu \in \mathbb{R}_+$ is the viscosity coefficient, and $f \in L^2_{\text{per}}((0,l)^2;\mathbb{R})$ is the forcing function. We want to learn the operator mapping the vorticity from the initial condition to the full solution




(a) KF: vorticity (b) Cavity: velocity in x, y

Figure 4: PINO on Kolmogorov flow (left) and Lid-cavity flow (right)

$\mathcal{G}^\dagger : w_0 \mapsto w|_{[0,T]}$.

$$\begin{aligned} \partial_t w(x,t) + u(x,t)\cdot\nabla w(x,t) &= \nu\Delta w(x,t) + f(x), && x \in (0,l)^2, t \in (0,T] \\ \nabla\cdot u(x,t) &= 0, && x \in (0,l)^2, t \in [0,T] \\ w(x,0) &= w_0(x), && x \in (0,l)^2 \end{aligned} \tag{20}$$

Specially, we consider two problem settings:

- **Long temporal transient flow**: we study the build-up of the flow from the initial condition $u_0$ near-zero velocity to $u_T$ that reaches the ergodic state. We choose $t \in [0, 50]$, $l = 1$, $Re = 20$ as in **(author?)** [23]. The main challenge is to predict the long time interval.
- **Chaotic Kolmogorov flow**: In this case $u$ lies in the attractor where arbitrary starting time $t_0$. We choose $t \in [t_0, t_0 + 0.5]$ or $[t_0, t_0 + 1]$, $l = 2\pi$, $Re = 500$ similar to **(author?)** [26]. The main challenge is to capture the small details that evolve chaotically.
- **Lid cavity flow**: In this case, we assume the no-slip boundary condition where $u(x, t) = (0, 0)$ at left, bottom, and right walls and $u(x, t) = (1, 0)$ on top, similar to **(author?)** [1]. We choose $t \in [5, 10]$, $l = 1$, $Re = 500$. The main challenge is to address the boundary using the velocity-pressure formulation.

## 4.1 Operator learning with physics constraints

We show that we can utilize the equation constraints to improve neural operator training. For this purpose, we train neural operators on fixed-resolution data in the presence of physics loss, $\mathcal{J}_{pde}$, and test the performance of the trained operators on high-resolution data. In particular, we test the performance of the trained model on data with the same resolution of the training data, 1x, 2x, and 4x, of the training data resolution Table 1. We observe that incorporating the $\mathcal{J}_{pde}$ in the training results in operators that, with high accuracy, generalize across data resolution. In this experiment, the training data for Burgers equation setting is in $32 \times 25$ (spatio-temporal) and the $\mathcal{J}_{pde}$ is imposed in $128 \times 100$ resolution. We use 800 low-resolution data and the same 800 PDE instances. The mean relative $L_2$ error and its standard deviation are reported over 200 test instances at resolution $32 \times 25$, $64 \times 50$, and $128 \times 100$.

Accordingly, the training data for the Darcy equation setting is at the spatial resolution of $11 \times 11$ and the $\mathcal{J}_{pde}$ is imposed in $61 \times 61$ resolution. We use 1000 low-resolution data and the same 1000 PDE instances. The mean and standard error are reported over 500 test instances at resolution $11 \times 11$, $61 \times 61$, and $211 \times 211$. Darcy flow

is unresolved at the $11 \times 11$ resolution, training on such a coarse grid causes higher errors. However, adding higher resolution PDE loss helps the operator to resolve.

The training data for Kolmogorov flow is in $64 \times 64 \times 33$ and the $\mathcal{J}_{pde}$ is imposed in $256 \times 256 \times 65$ resolution for the time interval $[0, 0.125]$. We use 800 low-resolution data and 2200 PDE instances. The mean and std of the relative $L_2$ error are reported over 200 test instances, at resolution $64 \times 64 \times 33$, $128 \times 128 \times 33$, and $256 \times 256 \times 65$.

| PDE | Training setting | Error at data resolution | Error at 2+ data resolution | Error at 4+ data resolution |
|---|---|---|---|---|
| Burgers | Data | 0.32±0.01% | 3.32±0.02% | 3.76±0.02% |
| | Data & PDE loss | 0.17±0.01% | 0.28±0.01% | 0.38±0.01% |
| Darcy | Data | 5.41±0.12% | 9.01±0.07% | 9.46±0.07% |
| | Data & PDE loss | 5.23±0.12% | 1.56±0.05% | 1.58±0.06% |
| Kolmogorov flow | Data | 8.28%±0.15% | 8.27%±0.15% | 8.30%±0.15% |
| | Data & PDE loss | 6.04%±0.12% | 6.02%±0.12% | 6.01%±0.12% |

Table 1: Operator-learning using fixed resolution data and PDE loss allows to train operators with high accuracy on very high resolution unseen data.

**Burgers equation and Darcy equation.** PINO can learn the solution operator without any data on simpler problems such as Burgers and Darcy. Compared to other PDE-constrained operators, PINO is more expressive and thereby achieves better accuracy. On Burgers (18), PI-DeepONet achieves **1.38%** [41]; PINO achieves **0.38%**. Similarly, on Darcy flow (19), PINO outperforms FNO by utilizing physics constraints, as shown in Table 2. For these simpler equations, instance-wise fine-tuning may not be needed. The implementation detail and the search space of parameters are included in Appendix A.1 and A.2.

| Method | Solution error |
|---|---|
| DeepONet with data [29] | $6.97 \pm 0.09\%$ |
| PINO with data | $1.22 \pm 0.03\%$ |
| PINO w/o data | $1.50 \pm 0.03\%$ |

Table 2: Operator learning on Darcy Flow equation. Incorporating physics constraints in operator learning improves the performance of neural operators.

| # data samples | # PDE instances | Solution error |
|---|---|---|
| 0 | 2200 | 6.22%±0.11% |
| 800 | 2200 | 6.01%±0.12% |
| 2200 | 2200 | 5.04%±0.11% |

Table 3: Physics-informed neural operator learning on Kolmogorov flow $Re = 500$. PINO is effective and flexible in combining physics constraints and any amount of available data. The mean and standard error of the relative $L_2$ test error is reported over 200 instances and evaluated on resolution $256 \times 256 \times 65$.

**Chaotic Kolmogorov flow.** We conduct an empirical study on how PINO can improve the generalization of neural operators by enforcing more physics. In the first experiment, we consider the Kolmogorov flow with $T = 0.125$. We train PINO with 2200 initial conditions and different amounts of low-resolution data. As shown in Table 3, PINO achieves $6.22\%$ error even without any data. We also observe that adding more low-resolution data to training makes the optimization easier and consistently improves the accuracy of the learned operator, showing that PINO is effective and flexible in combining physics constraints and any amount of available data.

The second experiment considers the Kolmogorov flow with $T = 0.5$. The training set consists of 4000 data points of the initial condition and corresponding solution. For operator learning, we sample high-resolution initial conditions

from a Gaussian random field. Table 7 compare the generalization error of neural operators trained by different schemes and different amounts of simulated data. The result shows that training neural operator with additional PDE instances consistently improves the generalization error on all three resolutions we are evaluating. Note that the relative $L_2$ error in this setting is much higher than the first one because the time horizon is 4 times longer. Next, we show how to solve for specific instances by finetuning the learned operator.

## 4.2 Solve equation using operator ansatz

We solve specific equation instances by fine-tuning the learned operator ansatz.

**Long temporal transient flow.** It is extremely challenging to propagate the information from the initial condition to future time steps over such a long interval $T = [0, 50]$ just using the soft physics constraint. Neither the PINN nor PINO (from scratch) can handle this case (error $> 50\%$), no matter solving the full interval at once or solving per smaller steps. However, when the data is available for PINO, we can use the learned neural operator ansatz and the anchor loss $\mathcal{L}_{op}$. The anchor loss is a hard constraint that makes the optimization much easier. Providing $N = 4800$ training data, the PINO without instance-wise fine-tuning achieves **2.87%** error, lower than FNO **3.04%** and it retains a 400x speedup compared to the GPU-based pseudo-spectral solver [11], matching FNO. Further doing test time optimization with the anchor loss and PDE loss, PINO reduces the error to **1.84%**.

**Chaotic Kolmogorov flow.** Based on the solution operators learned in Section 4.1, the second operator-learning setting, we continue to do instance-wise fine-tuning. We compare our method against other physics-informed learning methods including PINN [35], LAAF-PINN [15], and SA-PINN [32], as shown in Figure 3 and Table 4. Overall, PINO outperforms PINN and its improved variants by **20x** smaller error and **25x** speedup. Using a learned operator model makes PINO converge faster. The implementation detail and the search space of parameters are included in Appendix A.4.

| Method | # data samples | # PDE instances | Solution error $(w)$ | Time cost |
|---|---|---|---|---|
| PINNs | - | - | 18.7% | 4577s |
| PINO | 0 | 0 | **0.9%** | 608s |
| PINO | 0.4k | 0 | **0.9%** | 536s |
| PINO | 0.4k | 160k | **0.9%** | **473**s |

Table 4: Instance-wise fine-tuning on Kolmogorov flow $Re = 500, T = 0.5$. Using the learned operator as the initial condition helps fine-tuning converge faster.

**Transfer Reynolds numbers.** The extrapolation of different parameters and conditions is one of the biggest challenges for ML-based methods. It poses a domain shift problem. In this experiment, we train the source operator model on one Reynolds number and then fine-tune the model to another Reynolds number, on the Kolmogorov flow with $T = 1$. As shown in Table 8 by doing instance-wise fine-tuning, PINO can be easily transferred to different Reynolds numbers ranging from $100$ to $500$. This transferability shows PINO learned the dynamics shared across different Reynolds numbers. Such property envisions broad applications including transferring the learned operator to different boundary conditions or geometries.

**Lid cavity flow.** We demonstrate an additional example using PINO to solve for lid-cavity flow on $T = [5, 10]$ with $Re = 500$. In this case, we do not have the operator-learning phase and directly solve the equation (instance-wise fine-tuning). We use PINO with the velocity-pressure formulation and resolution $65 \times 65 \times 50$ plus the Fourier numerical gradient. It takes 2 minutes to achieve a relative error of $14.52\%$. Figure 4 shows the ground truth and prediction of the velocity field at $t = 10$ where the PINO accurately predicts the ground truth. The experiment shows that PINO can address non-periodic boundary conditions and multiple output fields.

**Convergence of accuracy with respect to resolution.** We study the convergence rate of PINO in the instance-wise optimization setting, where we minimize the PDE loss under different resolutions without any data. For PINO, using a higher resolution is more effective compared to running gradient descent for longer iterations. We test PINO on the Kolmogorov flow with Re = 500 and T = 0.125. We use the Fourier method in the spatial dimension and the finite difference method in the temporal dimension. As shown in Table 5, PINO shares the same convergence rate of its differentiation methods with no obvious limitation from optimization. It has a first-order convergence rate in time when $dx$ is fine enough and an exponential convergence rate when $dt$ is fine enough. It implies the PDE constraint can achieve high accuracy given a reasonable computational cost, and the virtual instances are almost as good as the data instances generated by the solver. Since the PDE loss can be computed on an unlimited amount of virtual instances in the operator learning setting, it is possible to reduce the generalization error going to zero by sampling virtual instances.

| dx \ dt | $2^{-6}$ | $2^{-7}$ | $2^{-8}$ | $2^{-9}$ | $2^{-10}$ |
|---|---|---|---|---|---|
| $2^{-4}$ | 0.4081 | 0.3150 | 0.3149 | 0.3179 | 0.3196 |
| $2^{-5}$ | 0.1819 | 0.1817 | 0.1780 | 0.1773 | 0.1757 |
| $2^{-6}$ | 0.0730 | 0.0436 | 0.0398 | 0.0386 | 0.0382 |
| $2^{-7}$ | 0.0582 | 0.0234 | 0.0122 | 0.0066 | 0.0034 |

Table 5: relative L2 error of PINO (Finite-difference in time) on Kolmogorov flow with $Re = 500$ and $T = 0.125$. PINO inherits the convergence rate of its differentiation method with no limitation of optimization

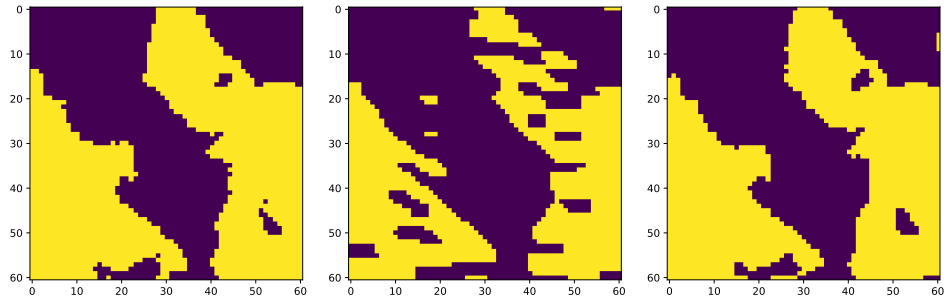
## 4.3 Inverse problem

One of the major advantages of the physics-informed method is to solve the inverse problem. In the experiment, we investigate PINO on the inverse problem of the Darcy equation to recover the coefficient function $a^\dagger$ from the given solution function $u^\dagger$. We assume a dataset $\{a_j, u_j\}$ is available to learn the operator. The coefficient function $a$ is a piecewise constant (representing two types of media), so the inverse problem can be viewed as a classification problem. We define $R(a)$ as the total variance.

The PDE loss strongly improves the prediction of the inverse problem. The plain neural operator model, while accurate in the forward problem, is vulnerable under perturbation and shift of the input $a$, which is a common behavior of deep-learning models. This domain-shift problem is critical for optimization-based inverse problems. During the optimization, $a$ is likely to go out of the training distribution, which makes the neural operator model inaccurate. As shown in Figure 5 (b), the prediction of $a$ is less accurate, while the model believes its output 5 (e) is the same as the target. This issue is mitigated by adding the PDE constraints, which restrict the prediction $a$ to the physically-valid manifold where $\mathcal{P}(a, u) = 0$. As shown in Figure 5 (c), the initial condition recovered with PDE loss is very close to the ground truth.

Comparing the PINO forward model with the inverse model, the inverse model $\mathcal{F}_\theta : u \mapsto a$ (17) has the best performance. As Shown in Figure 6, the inverse model has **2.29%** relative l2 error on the output $u$ and **97.10%** classification accuracy on the input $a$; the forward model has 6.43% error on the output and 95.38% accuracy on the input. Both models converge with 200 iterations. The major advantage of the PINO inverse model compared to the PINO forward model is that it uses a neural operator $\mathcal{F}_\theta(u^\dagger)$ as the ansatz for the coefficient function, which is used as regularization $\mathcal{L}_{op}$. Similar to the forward problem, the operator ansatz has an easier optimization landscape while being expressive.

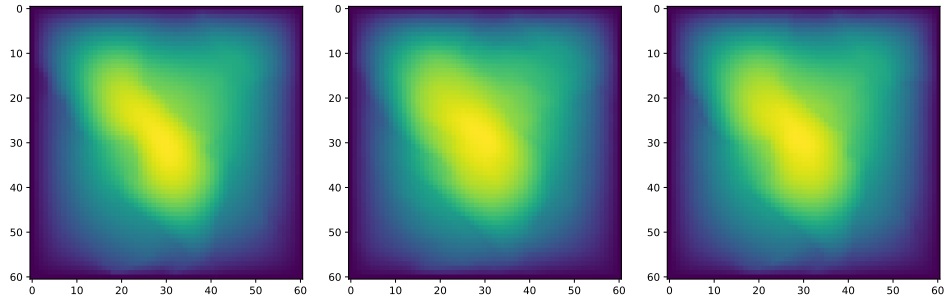
As a reference, we compare the PINO inverse frameworks with PINN and the conventional solvers using the accelerated MCMC method with 500,000 steps [5]. The posterior mean of the MCMC has a 4.52% error and 90.30% respectively ( Notice the Bayesian method outputs the posterior distribution, which is beyond obtaining a maximum a posteriori estimation). Meanwhile, PINO methods are 3000x faster compared to MCMC. PINN does not converge in this case. Please refer to D for further study on the importance of imposing physics constraints in approaching inverse problems in PDE.



(a) Ground truth input $a$

(b) Inversion using only data constraint

(c) Inversion using data and PDE constraints



(d) Observed output function

(e) Output function of inversion using only data constraint

(f) Output function of inversion using data and PDE constraints

Figure 5: In the above figures, (5a) represents the ground truth input function $a^\dagger$, and (5d) demonstrates the corresponding solution $u^\dagger$, i.e., the output function. Given the output $u^\dagger$, we aim to recover what $a$ could have generated the output function $u^\dagger$. Using only data constraint, (5b) shows that our method can find an $a$ that results in an output function very close to the ground truth $u^\dagger$ (5e). However, the recovered $a$ is far from satisfying the PDE equation. Using both data and PDE constraints, (5c) shows that our physics-informed method can find an $a$ that not only results in an output function very close to the ground truth $u^\dagger$(5f), but also the recovered $a$ satisfies the PDE constraint and is close to the underlying $a^\dagger$.

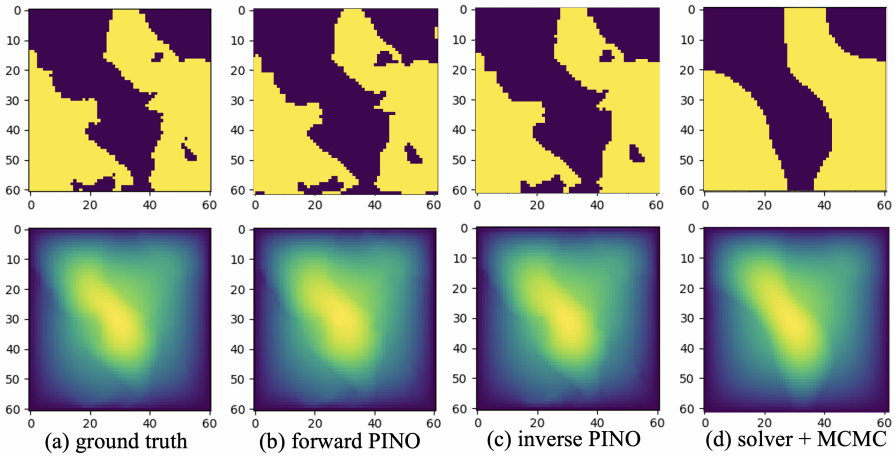



Figure 6: Darcy inverse problem: comparing PINO forward, inverse models with numerical solver with MCMC.

# 5 Conclusion and future work

In this work, we develop the physics-informed neural operator (PINO) that bridges the gap between physics-informed optimization and data-driven neural operator learning. We introduce the operator-learning and instance-wise fine-tuning schemes for PINO to utilize both the data and physics constraints. In the operator learning phase, PINO learns an operator ansatz over multiple instances of a parametric PDE family. The instance-wise fine-tuning scheme allows us to take advantage of the learned neural operator ansatz and solve for the solution function on the querying instance faster and more accurately.

There are many exciting future directions. Most of the techniques and analysis of PINN can be transferred to PINO. It is also interesting to ask how to overcome the hard trade-off of accuracy and complexity, and how the PINO model transfers across different geometries. Furthermore, it is promising to develop a software library of pre-trained models. PINO's excellent extrapolation property allows it to be applied on a broad set of conditions, as shown in the Transfer Reynold's number experiments.

# Acknowledgement

The authors want to thank Sifan Wang for meaningful discussions.

Z. Li gratefully acknowledges the financial support from the Kortschak Scholars, PIMCO Fellows, and Amazon AI4Science Fellows programs. N. Kovachki is partially supported by the Amazon AI4Science Fellowship. A. Anandkumar is supported in part by Bren endowed chair professorship.

This work was carried out on (1) the NVIDIA NGC as part of Zongyi Li's internship and (2) the Resnick High Performance Computing Center, a facility supported by Resnick Sustainability Institute at the California Institute of Technology

# References

[1] Charles-Henri Bruneau and Mazen Saad. The 2d lid-driven cavity problem revisited. *Computers & fluids*, 35(3):326–348, 2006.

[2] Oscar P Bruno, Youngae Han, and Matthew M Pohlman. Accurate, high-order representation of complex three-dimensional surfaces via fourier continuation analysis. *Journal of computational Physics*, 227(2):1094–1125, 2007.

[3] Steven L Brunton, Bernd R Noack, and Petros Koumoutsakos. Machine learning for fluid mechanics. *Annual Review of Fluid Mechanics*, 52:477–508, 2020.

[4] Shengze Cai, Zhiping Mao, Zhicheng Wang, Minglang Yin, and George Em Karniadakis. Physics-informed neural networks (pinns) for fluid mechanics: A review. *arXiv preprint arXiv:2105.09506*, 2021.

[5] Simon L Cotter, Gareth O Roberts, Andrew M Stuart, and David White. Mcmc methods for functions: modifying old algorithms to make them faster. *Statistical Science*, 28(3):424–446, 2013.

[6] Vikas Dwivedi and Balaji Srinivasan. Physics informed extreme learning machine (pielm)–a rapid method for the numerical solution of partial differential equations. *Neurocomputing*, 391:96–118, 2020.

[7] Olga Fuks and Hamdi A Tchelepi. Limitations of physics informed machine learning for nonlinear two-phase transport in porous media. *Journal of Machine Learning for Modeling and Computing*, 1(1), 2020.

[8] Han Gao, Luning Sun, and Jian-Xun Wang. Phygeonet: physics-informed geometry-adaptive convolutional neural networks for solving parameterized steady-state pdes on irregular domain. *Journal of Computational Physics*, 428:110079, 2021.

[9] Daniel Greenfeld, Meirav Galun, Ronen Basri, Irad Yavneh, and Ron Kimmel. Learning to optimize multigrid pde solvers. In *International Conference on Machine Learning*, pages 2415–2423. PMLR, 2019.

[10] Jiequn Han, Arnulf Jentzen, and E Weinan. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510, 2018.

[11] Yinnian He and Weiwei Sun. Stability and convergence of the crank–nicolson/adams–bashforth scheme for the time-dependent navier–stokes equations. *SIAM Journal on Numerical Analysis*, 45(2):837–869, 2007.

[12] Oliver Hennigh, Susheela Narasimhan, Mohammad Amin Nabian, Akshay Subramaniam, Kaustubh Tangsali, Zhiwei Fang, Max Rietmann, Wonmin Byeon, and Sanjay Choudhry. Nvidia simnet™: An ai-accelerated multi-physics simulation framework. In *International Conference on Computational Science*, pages 447–461. Springer, 2021.

[13] Hans Hersbach, Bill Bell, Paul Berrisford, Shoji Hirahara, András Horányi, Joaquín Muñoz-Sabater, Julien Nicolas, Carole Peubey, Raluca Radu, Dinand Schepers, et al. The era5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730):1999–2049, 2020.

[14] Xiang Huang, Zhanhong Ye, Hongsheng Liu, Beiji Shi, Zidong Wang, Kang Yang, Yang Li, Bingya Weng, Min Wang, Haotian Chu, et al. Meta-auto-decoder for solving parametric partial differential equations. *arXiv preprint arXiv:2111.08823*, 2021.

[15] Ameya D Jagtap, Kenji Kawaguchi, and George Em Karniadakis. Locally adaptive activation functions with slope recovery for deep and physics-informed neural networks. *Proceedings of the Royal Society A*, 476(2239):20200334, 2020.

[16] Xiaowei Jin, Shengze Cai, Hui Li, and George Em Karniadakis. Nsfnets (navier-stokes flow nets): Physics-informed neural networks for the incompressible navier-stokes equations. *Journal of Computational Physics*, 426:109951, 2021.

[17] K Kashinath, M Mustafa, A Albert, JL Wu, C Jiang, S Esmaeilzadeh, K Azizzadenesheli, R Wang, A Chattopadhyay, A Singh, et al. Physics-informed machine learning: case studies for weather and climate modelling. *Philosophical Transactions of the Royal Society A*, 379(2194):20200093, 2021.

[18] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[19] Dmitrii Kochkov, Jamie A Smith, Ayya Alieva, Qing Wang, Michael P Brenner, and Stephan Hoyer. Machine learning accelerated computational fluid dynamics. *arXiv preprint arXiv:2102.01010*, 2021.

[20] Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces. *arXiv preprint arXiv:2108.08481*, 2021.

[21] Samuel Lanthaler, Roberto Molinaro, Patrik Hadorn, and Siddhartha Mishra. Nonlinear reconstruction for operator learning of pdes with discontinuities. *arXiv preprint arXiv:2210.01074*, 2022.

[22] Zongyi Li, Daniel Zhengyu Huang, Burigede Liu, and Anima Anandkumar. Fourier neural operator with learned deformations for pdes on general geometries. *arXiv preprint arXiv:2207.05209*, 2022.

[23] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations, 2020.

[24] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Multipole graph neural operator for parametric partial differential equations, 2020.

[25] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Graph kernel network for partial differential equations. *arXiv preprint arXiv:2003.03485*, 2020.

[26] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Markov neural operators for learning chaotic systems. *arXiv preprint arXiv:2106.06898*, 2021.

[27] Burigede Liu, Nikola Kovachki, Zongyi Li, Kamyar Azizzadenesheli, Anima Anandkumar, Andrew M Stuart, and Kaushik Bhattacharya. A learning-based multiscale method and its application to inelastic impact problems. *Journal of the Mechanics and Physics of Solids*, 158:104668, 2022.

[28] Denghui Lu, Han Wang, Mohan Chen, Lin Lin, Roberto Car, E Weinan, Weile Jia, and Linfeng Zhang. 86 pflops deep potential molecular dynamics simulation of 100 million atoms with ab initio accuracy. *Computer Physics Communications*, 259:107624, 2021.

[29] Lu Lu, Pengzhan Jin, and George Em Karniadakis. Deeponet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators. *arXiv preprint arXiv:1910.03193*, 2019.

[30] Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data. *arXiv preprint arXiv:2111.05512*, 2021.

[31] Lu Lu, Xuhui Meng, Zhiping Mao, and George Em Karniadakis. DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, 63(1):208–228, 2021.

[32] Levi McClenny and Ulisses Braga-Neto. Self-adaptive physics-informed neural networks using a soft attention mechanism. *arXiv preprint arXiv:2009.04544*, 2020.

[33] Jaideep Pathak, Mustafa Mustafa, Karthik Kashinath, Emmanuel Motheau, Thorsten Kurth, and Marcus Day. Ml-pde: A framework for a machine learning enhanced pde solver. *Bulletin of the American Physical Society*, 2021.

[34] Jaideep Pathak, Shashank Subramanian, Peter Harrington, Sanjeev Raja, Ashesh Chattopadhyay, Morteza Mardani, Thorsten Kurth, David Hall, Zongyi Li, Kamyar Azizzadenesheli, et al. Fourcastnet: A global data-driven high-resolution weather model using adaptive fourier neural operators. *arXiv preprint arXiv:2202.11214*, 2022.

[35] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[36] Maziar Raissi, Alireza Yazdani, and George Em Karniadakis. Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations. *Science*, 367(6481):1026–1030, 2020.

[37] Justin Sirignano and Konstantinos Spiliopoulos. Dgm: A deep learning algorithm for solving partial differential equations. *Journal of computational physics*, 375:1339–1364, 2018.

[38] Jonathan D Smith, Zachary E Ross, Kamyar Azizzadenesheli, and Jack B Muir. Hyposvi: Hypocenter inversion with stein variational inference and physics informed neural networks. *arXiv*, 2021.

[39] Luning Sun, Han Gao, Shaowu Pan, and Jian-Xun Wang. Surrogate modeling for fluid flows based on physics-constrained deep learning without simulation data. *Computer Methods in Applied Mechanics and Engineering*, 361:112732, 2020.

[40] Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021.

[41] Sifan Wang, Hanwen Wang, and Paris Perdikaris. Learning the solution operator of parametric partial differential equations with physics-informed deeponets. *arXiv preprint arXiv:2103.10974*, 2021.

[42] Sifan Wang, Xinling Yu, and Paris Perdikaris. When and why pinns fail to train: A neural tangent kernel perspective. *arXiv preprint arXiv:2007.14527*, 2020.

[43] E Weinan and Bing Yu. The deep ritz method: a deep learning-based numerical algorithm for solving variational problems. *Communications in Mathematics and Statistics*, 6(1):1–12, 2018.

[44] Gege Wen, Zongyi Li, Qirui Long, Kamyar Azizzadenesheli, Anima Anandkumar, and Sally M Benson. Accelerating carbon capture and storage modeling using fourier neural operators. *arXiv preprint arXiv:2210.17051*, 2022.

[45] Haoyu Yang, Zongyi Li, Kumara Sastry, Saumyadip Mukhopadhyay, Mark Kilgard, Anima Anandkumar, Brucek Khailany, Vivek Singh, and Haoxing Ren. Generic lithography modeling with dual-band optics-inspired neural networks. *arXiv preprint arXiv:2203.08616*, 2022.

[46] Lulu Zhang, Tao Luo, Yaoyu Zhang, Zhi-Qin John Xu, and Zheng Ma. Mod-net: A machine learning approach via model-operator-data network for solving pdes. *arXiv preprint arXiv:2107.03673*, 2021.

[47] Yinhao Zhu, Nicholas Zabaras, Phaedon-Stelios Koutsourelakis, and Paris Perdikaris. Physics-constrained deep learning for high-dimensional surrogate modeling and uncertainty quantification without labeled data. *Journal of Computational Physics*, 394:56–81, 2019.

# A Implementation details

In this section, we list the detailed experiment setups and parameter searching for each experiment in Section 4. Without specification, we use Fourier neural operator backbone with $width = 64$, $mode = 8$ or 12, $L = 4$ and GeLU activation. The numerical experiments are performed on Nvidia V100 GPUs and A100 GPUs.

## A.1 Burgers Equation

We use the 1000 initial conditions $u_0 \sim \mu$ where $\mu = \mathcal{N}(0, 625(-\Delta + 25I)^{-2})$ to train the solution operator on PINO with $width = 64$, $mode = 15$, and GeLU activation. We use the numerical method to take the gradient. We use Adam optimizer with the learning rate $0.001$ that decays by half every $100$ epochs. $500$ epochs in total. The total training time is about $1250s$ on a single Nvidia 3090 GPU. PINO achieves **0.38%** relative l2 error averaged over 200 testing instances. PINN-DeepONet achieves **1.38%** which is taken from **(author?)** [41] which uses the same problem setting.

## A.2 Darcy Flow

We use the $1000$ coefficient conditions $a$ to train the solution operator where $a \sim \mu$ where $\mu = \psi_{\#}\mathcal{N}(0, (-\Delta + 9I)^{-2})$, $\psi(a(x)) = 12$ if $a(x) \geq 0$; $\psi(a(x)) = 3$ if $a(x) < 0$. The zero boundary condition is enforced by multiplying a mollifier $m(x) = \sin(\pi x)\sin(\pi y)$ for all methods. The parameter of PINO on Darcy Flow is the same as in the Burgers equation above. Regarding the implementation detail of the baselines: as for FNO, we use the same hyperparameters as its paper did [25] and set $width = 64, mode = 20, depth = 4$; DeepONet [29] did not study Darcy flow so we grid search the hyperparameters of DeepONet: depth from 2 to 12, width from 50 to 100. The best result of DeepONet is achieved by depth 8, width 50. The results are shown in Table 2. All the models are trained on resolution $61 \times 61$ and evaluated on resolution $61 \times 61$.

## A.3 Long temporal transient flow.

We study the build-up of the flow from the initial condition $u_0$ near-zero velocity to $u_T$ that reaches the ergodic state. We choose $T = 50, l = 1$ as in **(author?)** [23]. We choose the weight parameters of error $\alpha = \beta = 5$. The initial condition $w_0(x)$ is generated according to $w_0 \sim \mu$ where $\mu = \mathcal{N}(0, 7^{3/2}(-\Delta + 49I)^{-2.5})$ with periodic boundary conditions. The forcing is kept fixed $f(x) = 0.1(\sin(2\pi(x_1 + x_2)) + \cos(2\pi(x_1 + x_2)))$. We compare FNO, PINO (no instance-wise fine-tuning), and PINO (with instance-wise fine-tuning). They get $3.04\%$, $2.87\%$, and $1.84\%$ relative l2 error on the last time step $u(50)$ over 5 testing instance.

## A.4 Chaotic Kolmogorov flow.

For this experiment, $u$ lies in the attractor. We choose $T = 0.125, 0.5$ or $1$, and $l = 1$, similar to **(author?)** [26]. For $T = 0.5s$, the training set consists of 4000 initial condition functions and corresponding solution functions with a spatial resolution of $64 \times 64$ and a temporal resolution of $65$. Extra initial conditions are generated from Gaussian random field $\mathcal{N}\left(0, 7^{3/2}(-\Delta + 49I)^{-5/2}\right)$. We estimate the generalization error of the operator on a test set that contains 300 instances of Kolmogorov flow and reports the averaged relative $L_2$ error. Each neural operator is trained with 4k data points plus a number of extra sampled initial conditions. The Reynolds number in this problem is 500. The reported generalization error is averaged over 300 instances. For the experiments in Table 1, $T = 0.125$ and the training set has data with spatial resolution $64 \times 64$ and temporal resolution 33. The test set has 200 instances with spatial resolution $256 \times 256$ and temporal resolution $65$. For the experiments in Table 6, $T = 0.125$ and the training set contains 800 instances in even lower resolution $32 \times 32 \times 17$. The test set has 200 instances with spatial resolution $256 \times 256$ and temporal resolution $65$.

**Comparison study.** The baseline method PINN, LAAF-PINN, SA-PINN are implemented using library DeepXDE [31] with TensorFlow as backend. We use the two-step optimization strategy (Adam [18] and L-BFGS) following the same practice as NSFNet [16], which applies PINNs to solving Navier Stokes equations. We grid search the hyperparameters: network depth from 4 to 6, width from 50 to 100, learning rate from 0.01 to 0.0001, the weight of boundary loss from 10 to 100 for all experiments of PINNs. Comparison between PINO and PINNs on instance-wise fine-tuning. The results are averaged over 20 instances of the Navier Stokes equation with Reynolds number 500. The

best result is obtained by PINO using learned operator ansatz and virtual sampling. The neural operator ansatz used here is trained over a set of 400 data points. The authors acknowledge that there could exist more sophisticated variants of PINN that performs better in our test cases.

| Test resolution | FNO | PINO |
|---|---|---|
| 64x64x33 | 9.73± 0.15% | 6.30±0.11% |
| 128x128x33 | 9.74± 0.16% | 6.28±0.11% |
| 256x256x65 | 9.84± 0.16% | 6.22±0.11% |

Table 6: Comparison between data only (FNO) and data + pde (PINO) on higher resolutions while trained on much lower resolution $32 \times 32 \times 17$.

| # data samples | # additional PDE instances | Resolution | Solution error | Equation error |
|---|---|---|---|---|
| 400 | 0 | $128 \times 128 \times 65$ | 33.32% | 1.8779 |
| | | $64 \times 64 \times 65$ | 33.31% | 1.8830 |
| | | $32 \times 32 \times 33$ | 30.61% | 1.8421 |
| 400 | 40k | $128 \times 128 \times 65$ | 31.74% | 1.8179 |
| | | $64 \times 64 \times 65$ | 31.72% | 1.8227 |
| | | $32 \times 32 \times 33$ | 29.60% | 1.8296 |
| 400 | 160k | $128 \times 128 \times 65$ | 31.32% | 1.7840 |
| | | $64 \times 64 \times 65$ | 31.29% | 1.7864 |
| | | $32 \times 32 \times 33$ | 29.28% | 1.8524 |
| 4k | 0 | $128 \times 128 \times 65$ | 25.15% | 1.8223 |
| | | $64 \times 64 \times 65$ | 25.16% | 1.8257 |
| | | $32 \times 32 \times 33$ | 21.41% | 1.8468 |
| 4k | 100k | $128 \times 128 \times 65$ | 24.15% | 1.6112 |
| | | $64 \times 64 \times 65$ | 24.11% | 1.6159 |
| | | $32 \times 32 \times 33$ | 20.85% | 1.8251 |
| 4k | 400k | $128 \times 128 \times 65$ | 24.22% | 1.4596 |
| | | $64 \times 64 \times 65$ | 23.95% | 1.4656 |
| | | $32 \times 32 \times 33$ | 20.10% | 1.9146 |
| 0 | 100k | $128 \times 128 \times 65$ | 74.36% | 0.3741 |
| | | $64 \times 64 \times 65$ | 74.38% | 0.3899 |
| | | $32 \times 32 \times 33$ | 74.14% | 0.5226 |

Table 7: Each neural operator is trained with 400 or 4k data points additionally sampled free initial conditions. The Reynolds number is 500. The reported generalization error is averaged over 300 instances. Training on additional initial conditions boosts the generalization ability of the operator.

## A.5 Transfer learning across Reynolds numbers

We study the instance-wise fine-tuning with different Reynolds numbers on the $T = 1$ Kolmogorov flow. For the higher Reynolds number problem $Re = 500, 400$, fine-tuning the source operator shows better convergence accuracy compared to learning from scratch. In all cases, the fine-tuning the source operator shows better convergence speed as demonstrated in Figure 7. The results are shown in Table 8 where the error is averaged over 40 instances. Each row is a testing case and each column is a source operator.

| Testing Re | From scratch | 100 | 200 | 250 | 300 | 350 | 400 | 500 |
|---|---|---|---|---|---|---|---|---|
| 500 | 0.0493 | 0.0383 | 0.0393 | 0.0315 | 0.0477 | 0.0446 | 0.0434 | 0.0436 |
| 400 | 0.0296 | 0.0243 | 0.0245 | 0.0244 | 0.0300 | 0.0271 | 0.0273 | 0.0240 |
| 350 | 0.0192 | 0.0210 | 0.0211 | 0.0213 | 0.0233 | 0.0222 | 0.0222 | 0.0212 |
| 300 | 0.0168 | 0.0161 | 0.0164 | 0.0151 | 0.0177 | 0.0173 | 0.0170 | 0.0160 |
| 250 | 0.0151 | 0.0150 | 0.0153 | 0.0151 | 0.016 | 0.0156 | 0.0160 | 0.0151 |
| 200 | 0.00921 | 0.00913 | 0.00921 | 0.00915 | 0.00985 | 0.00945 | 0.00923 | 0.00892 |
| 100 | 0.00234 | 0.00235 | 0.00236 | 0.00235 | 0.00239 | 0.00239 | 0.00237 | 0.00237 |

Table 8: Reynolds number transfer learning. Each row is a test set of PDEs with corresponding Reynolds number. Each column represents the operator ansatz we use as the starting point of instance-wise fine-tuning. For example, column header "100" means the operator ansatz is trained over a set of PDEs with Reynolds number 100. The relative $L_2$ errors is averaged over 40 instances of the corresponding test set.

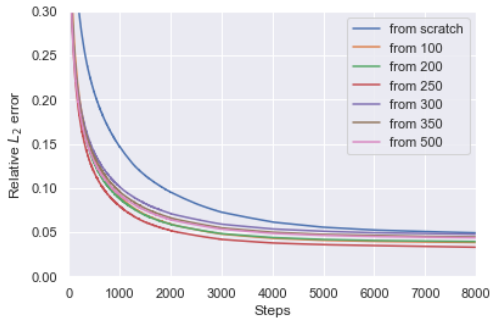
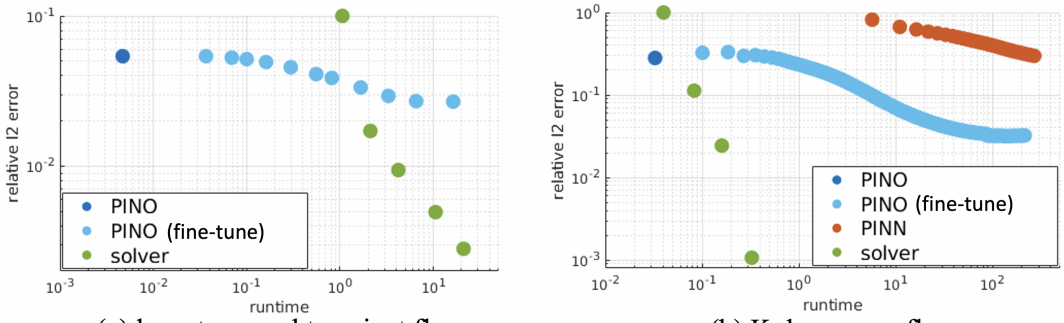



Figure 7: Plot of relative $L_2$ error versus update step for the Kolmogorov flow with Reynolds number $500$, $T = 1$. The test error is averaged over 40 instances. We observe that all the operator ansatzs trained over PDE instances with different Reynolds number can boost the instance-wise fine-tuning accuracy and convergence speed compared to training from scratch.




(a) long-temporal transient flow (b) Kolmogorov flow

(a) The long-temporal transient flow with $Re \sim 20, T = 50$. PINO outputs the full trajectory in one step, which leads to 400x speedup compared to the GPU solver. PINN cannot converge to a reasonable error rate due to the long time window. (b) The chaotic Kolmogorov flow with $Re =\sim 500, T = 0.5$. PINO converges faster compared to PINN, but their convergence rates with gradient descent are less effective compared to using higher resolutions in the GPU solver.

Figure 8: The accuracy-complexity trade-off on PINO, PINN, and the GPU-based pseudo-spectral solver.

# B Additional experiments

## B.1 Additional baselines

We add a comparison experiment against the Locally adaptive activation functions for PINN (LAAF-PINN) [15] and Self-Adaptive PINN (SA-PINN) [32]. For the Kolmogorov flow problem, we set Re=500, T=[0, 0.5]. We search among the following hyperparameters combinations: LAAF-PINN: n: 10, 100, learning rate: 0.1, 0.01, 0.001, depth 4, 6. SA-PINNs: learning rate 0.001, 0.005, 0.01, 0.05, network width 50, 100, 200, depth 4, 6, 8.

As shown in Figure 3, both LAAF-PINN and SA-PINN converge much faster compared to the original PINN method, but there is still a big gap with PINO. LAAF-PINN adds learnable parameters before the activation function; SA-PINN adds the weight parameter for each collocation point. These techniques help to alleviate the PINNs‘ optimization problem significantly. However, they didn't alter the optimization landscape effectively in the authors' opinion. On the other hand, by using operator ansatz, PINO optimizes in a function-wise manner where the optimization is fundamentally different.

Note that the contribution of PINO is orthogonal to the above methods. One can apply the adaptive activation functions or self-adaptive loss in the PINO framework too. All these techniques of PINNs can be straightforwardly transferred to PINO. We believe it would be interesting future directions to study how all these methods work with each other in different problems.

## B.2 Lid Cavity flow.

We demonstrate an addition example using PINO to solve for lid-cavity flow on $T = [5, 10]$ with $Re = 500$. In this case, we do not have the operator-learning phase and directly solve the equation (instance-wise fine-tuning). We use PINO with the velocity-pressure formulation and resolution $65 \times 65 \times 50$ plus the Fourier numerical gradient. It takes 2 minutes to achieve a relative error of $14.52\%$. Figure 4 shows the ground truth and prediction of the velocity field at $t = 10$ where the PINO accurately predicts the ground truth.

We assume a no-slip boundary where $u(x, t) = (0, 0)$ at left, bottom, and right walls and $u(x, t) = (1, 0)$ on top, similar to **(author?)** [1]. We choose $t \in [5, 10]$, $l = 1$, $Re = 500$. We use the velocity-pressure formulation as in **(author?)** [16] where the neural operator output the velocity field in $x$, $y$, and the pressure field. We set $width = 32$, $mode = 20$ with learning rate $0.0005$ which decreases by half every $5000$ iterations, $5000$ iterations in total. We use the Fourier method with Fourier continuation to compute the numerical gradient and minimize the residual error on the velocity, the divergence-free condition, as well as the initial condition and boundary condition. The weight parameters $(\alpha, \beta)$ between different error terms are all chosen as 1. Figure 4 shows the ground truth and prediction of the velocity field at $t = 10$ where the PINO accurately predicts the ground truth.

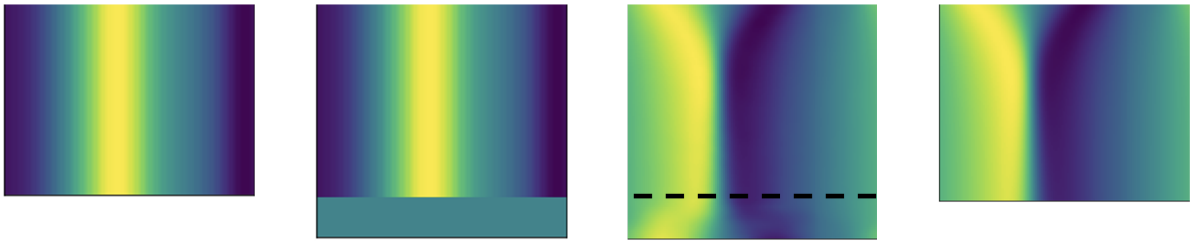



Figure 9: Fourier Continuation by padding zeros. The x-axis is spatial dimension; the y-axis is the temporal dimension. FNO extends the output smoothly on the padded domain.

# C Fourier continuation

The Fourier neural operator can be applied to arbitrary geometry via Fourier continuations. Given any compact manifold $\mathcal{M}$, we can always embed it into a periodic cube (torus),

$$i : \mathcal{M} \to \mathcal{T}^n$$

where we can do the regular FFT. Conventionally, people would define the embedding $i$ as a continuous extension by fitting polynomials [2]. However, in Fourier neural operator, it can be simply done by padding zeros in the input. The loss is computed at the original space during training. The Fourier neural operator will automatically generate a smooth extension to do padded domain in the output, as shown in Figure 9.

This technique is first used in the original Fourier neural operator paper [23] to deal with the time dimension in the Navier-Stokes equation. Similarly, **(author?)** [30] apply FNO with extension and interpolation on diverse geometries on the Darcy equation. In the work, we use Fourier continuation widely for non-periodic boundary conditions (Darcy, time dimension). We also added an example of lid-cavity to demonstrate that PINO can work with non-periodic boundary conditions.

Furthermore, this Fourier continuation technique helps to take the derivatives of the Fourier neural operator. Since the output of FNO is always on a periodic domain, the numerical Fourier gradient is usually efficient and accurate, except if there is shock (in this case, we will use the exact gradient method).

# D PDE Constraints in Inverse Problems

In subsection 3.4 we presented the study of inverse problems where we propose a new approach to tackle inverse problems. We propose to incorporate both data and PDE constraints for the inverse problem. The data constraint makes sure the recovered input function, when fed to the neural operator, results in an output that matches the observed data $u^\dagger$. The PDE constraint, imposed using $\mathcal{L}_{pde}$, is a crucial physics-based constraint that is the key to having an accurate approach for inverse problems. This constraint reinforces that the pairs of recovered input and their corresponding output are physical, in another word, they satisfy the underlying PDE. When the physics-based constraint is imposed, the search space of $(a, u)$ is confined to the solutions manifold of the PDE, i.e., pairs satisfying the underlying PDE.